Pressure, Not Parking Time: Reclaimable Prefix State in Agentic LLM Autoscaling

Anonymous Author(s)
Submission #xxx

Abstract

Agent programs alternate model invocations with tool calls. During a tool call, GPU utilisation and request queues can both be zero even though the program will soon reuse a long prefix. We initially assumed that native vLLM reserved that program's KV blocks and that KV-cache utilisation therefore exposed the parked state. A hardware validation falsified this premise. In vLLM 0.25.1, a completed request releases its blocks to a reclaimable prefix cache: an immediate reuse may be cheap, but the exported active-KV metric also reads zero and new work can evict the prefix.

We report the measurement, correct a trace-driven simulator, and re-evaluate eight autoscaling policies. On Qwen2.5-14B, a 32k-token cache miss adds 9.03 s relative to a hit, while all eight parked sessions produce zero GPU, running, waiting, and active-KV readings in 114 samples. A pressure experiment shows that a 16k prefix remains intact behind at most eight concurrent 4k contexts, is about 64.5% retained behind 16, and is gone behind 24; delays of 0–32 s have little independent effect. The same run calibrates batching ($k_{1/2} = 4.606$) and a 38.155 s process cold start.

The correction reverses the original policy result. Across six agentic points on the Azure coding trace, mean SLO attainment is 0.935 for HPA, 0.931 for KV-util, and 0.909 for a candidate policy that counts parked programs. Across four baseline controllers and four turn-count regimes, 97.03% of destroyed prefix tokens are due to pressure eviction and only 2.97% to scale-in. Guided by this result, a simple pressure-aware admission baseline eliminates pressure eviction and reaches 1.0 SLO attainment at 4–8× synthetic load using 36–41% fewer GPU-seconds than HPA, though it remains costlier than the static reference. The useful signal is pressure on valuable reclaimable prefixes, not idle time alone.

Keywords

LLM serving, agents, autoscaling, prefix cache, admission control

1 Introduction

An LLM agent is a program rather than a request. It invokes a model, runs an external tool, appends the result, and invokes the model again on an extended transcript [2]. Between invocations it uses no GPU. We call this interval *parked*. The accumulated prefix can still be valuable: losing it forces a re-prefill before the next turn.

This creates a tempting but incorrect mental model. If a serving engine pins a parked program's KV blocks, compute-side signals miss the program while a memory-side signal sees it. Scaling in then destroys hard-held state. Our original study encoded exactly that model and produced a strong result for a controller that counted parked programs.

We designed a four-part vLLM validation to measure the assumptions before using them in the paper. The experiment did what validation should do: it falsified the central one. Native vLLM returns a completed request's blocks to its free/reclaimable pool. Prefix caching may preserve their content, but the program has no reservation and vLLM's exported active-KV metric reports no allocation for it. During a tool call all three deployed signals can therefore read zero simultaneously:

- GPU utilisation sees no kernel executing for the program;
- queue/running metrics see no submitted request; and
- active-KV utilisation sees no blocks allocated to an active request.

The state is neither resident in the resource-accounting sense nor absent in the performance sense. It is an opportunistic, evictable option on a future cache hit. This distinction changes both the model and the control problem.

This paper makes three contributions. First, it measures the missing state, its price, and its survival under time and pressure on a real vLLM instance. Second, it corrects the simulator to separate active allocation from reclaimable prefix content and shows that pressure eviction accounts for 97.03% of baseline prefix destruction. Third, it reports that the original ParkAware mechanism no longer wins reliably and evaluates the data-directed alternative: couple admission and scale-out to cache pressure. The result is a reproducible falsification followed by a positive, deliberately simple design point, rather than a preserved claim after its premise failed.

2 Workload and Signals

2.1 Agentic serving

Context accumulates across an agent trajectory. Turn i contains prior model outputs and tool results, so a later prompt may be tens of thousands of tokens. Serving systems use paged KV management and prefix caching to avoid repeating that prefill [3]. Published coding-agent runs contain long trajectories; we therefore sweep up to 64 turns rather than treating every invocation as an independent chat request [10].

Microsoft's public inference dataset provides one conversation trace and one coding trace [1]. The coding trace contains 8,819 requests with a context-to-generation ratio of 73:1, index of dispersion 71.3, and peak-to-mean arrival rate 12.8. The conversation trace contains 19,366 requests, with the corresponding values 5.5:1, 4.0, and 1.8 (Figure 1). The coding trace is not a trace of tool calls, but its long-context,

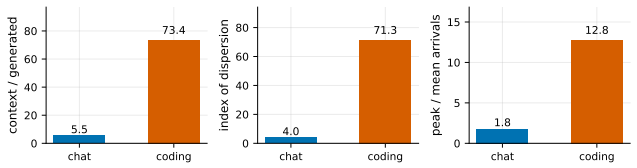

Figure 1: Measured properties of the two Azure traces.

short-output and bursty shape makes it a useful request-level substrate.

2.2 Three zeroes, one latent value

GPU utilisation, queue depth, and active-KV utilisation are sensible signals for submitted requests. A parked program is outside that interface. Yet a cache hit can make its next turn much cheaper. We denote a program's cached prefix by c_i and the fraction still present when it returns by $q_i \in [0, 1]$. Its latent recompute liability is

$$L_i = \frac{(1 - q_i)c_i}{R_{\text{prefill}}}. \tag{1}$$

Neither q_i nor the program's probability of returning is exported by the three native autoscaling signals. Counting programs alone supplies neither.

3 Hardware Validation

3.1 Setup

We ran vLLM 0.25.1 with prefix caching enabled, using the Qwen2.5-14B Instruct checkpoint on one device reported as RTX 4090 with 49,140 MiB. The server reported capacity for 75,216 KV-cache tokens. Each point has three repetitions unless noted. The repository contains the exact scripts, raw CSVs, server logs, and a machine-readable summary.

3.2 V1: price of a prefix

We issue the same prefix twice for a hit and a distinct prefix for a miss. At 1k tokens, the mean miss–hit penalty is 0.161 s; at 32k it is 9.028 s and the hit is 44.2× faster. A linear fit gives 0.286 s per 1k tokens with $R^2 = 0.989$. The state is valuable even though it is not reserved.

3.3 V2: the assumption that failed

Eight sessions alternate model calls with 8 s tool sleeps while DCGM and the vLLM Prometheus endpoint are sampled. Across 114 samples in which all sessions are parked, mean GPU utilisation, running requests, waiting requests, and active-KV utilisation are all exactly zero. When any session computes, the corresponding means are 93.5% GPU, 5.96 running, 0.62 waiting, and 29.5% active KV. Thus KV-util does not rescue observability of parked state.

3.4 V3–V4: calibration

At concurrency $k \in \{1, 2, 4, 8, 16, 32\}$, per-sequence output rate fits

$$r(k) = \frac{R}{k + k_{1/2}} \tag{2}$$

with $R = 156.19$, $k_{1/2} = 4.606$, and $R^2 = 0.99995$. The original assumed $k_{1/2} = 16$, so its batching model was materially too optimistic. Three clean server process starts with a warm host page cache take 38.163, 38.141, and 38.161 s (mean 38.155 s). This is not a disk-cold or 70B cold start.

3.5 P1: survival under delay and pressure

We park a 16k-token target, immediately issue 0, 4, 8, 16, or 24 distinct 4k-token neighbors, and then wait if necessary so the total park interval reaches the minimum target $\tau_{\min} \in \{0, 8, 32\}$ s before probing. We normalize probe TTFT between the same-trial hit and miss to estimate retained prefix fraction. All 45 trials completed. At $N \leq 8$, retention is essentially one; at $N = 16$, it is 0.645; and at $N = 24$, it is approximately zero. The pressure work itself takes about 12.2 s and 18.2 s at $N = 16$ and $N = 24$, respectively, so it overruns the 0/8-s minimum targets. Across the distinct elapsed times available at each pressure level, allocation pressure rather than wall-clock age determines survival in this range.

This result also exposes partial eviction. The measured capacity is 75,216 tokens; a 16k target plus sixteen 4k neighbors exceeds it by about 4.8k tokens, consistent with a partial rather than all-or-nothing miss.

4 Corrected Method

4.1 Real and constructed inputs

Arrival time, context length, and generated length come directly from the Azure traces. The traces contain neither program identifiers nor tool durations. We construct programs of $T \in \{1, 4, 16, 64\}$ turns and sweep mean tool delay $\tau \in \{0, 8, 30\}$ s. $T = 1, \tau = 0$ is the unmodified request-level control. Context accumulates across constructed turns. We use seed 20260720.

4.2 State model and service calibration

Each replica separately tracks active KV allocation and reclaimable cached content. A completed turn releases its active allocation but leaves its prefix as LRU content. New active blocks reclaim the oldest parked prefixes at token granularity. Scale-in removes only idle replicas and discards any cached content on them. On return, the program prefills only the missing part of its prefix.

Defaults use the measured 75,216-token capacity, $k_{1/2} = 4.606$, and 38.155 s process cold start. Jointly fitting V1 and V3 gives a service rate of 19,607 token-equivalents/s and decode weight 94.282 relative to prefill. Maximum batch is 32. E7 explicitly sweeps capacity, batch, and policy target.

We simulate 2,400 s of arrivals, exclude a 300 s warm-up from scoring, then stop arrivals and drain for the larger of

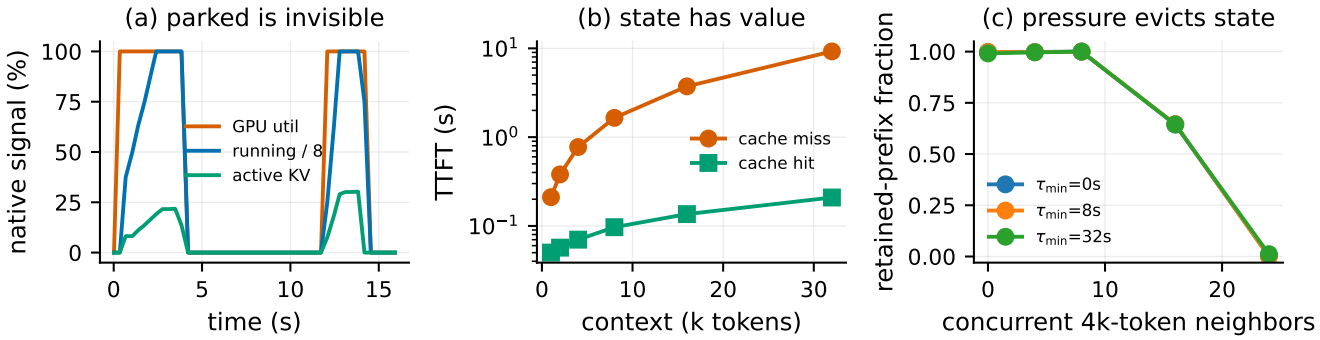

Figure 2: Hardware validation. (a) During tool-call intervals, all three native signals drop to zero. (b) Cache loss has a context-proportional TTFT cost. (c) A parked prefix survives elapsed time but not sufficient concurrent pressure; the score estimates the retained fraction from hit/miss TTFT.

600 s or a workload-derived bound. This drain is essential: scoring only to the last arrival mechanically labels late long programs as failures. Every admitted turn completes in the final runs, so goodput is one and does not separate policies.

The SLO is per-turn slowdown at most three. For each workload we search for the smallest static cluster achieving at least 95% SLO attainment. Dynamic cost is reported as GPU-seconds relative to that reference. Unfinished admitted work, if any, is counted as an SLO failure.

4.3 Policies

We implement HPA on GPU utilisation (70% target and 300 s scale-down stabilisation), KEDA on waiting requests, KV-util on active KV (80% target), a Holt arrival forecaster, and tabular Q-learning with and without parked count in its state [8, 9]. The candidate ParkAware policy uses

$$N = \max\left(\left\lceil \frac{\text{running} + \text{queued} + \text{parked}}{B\theta} \right\rceil, \left\lceil \frac{\text{active KV}}{K\theta} \right\rceil\right). \tag{3}$$

It is retained unchanged as the direct test of the original idea. Under the corrected mechanism, its parked term is a conservative seat reservation, not a measurement of pinned memory.

The new pressure-aware baseline instead admits at most eight active turns per replica, rejects any placement that would reclaim a parked prefix, and scales out on the larger of queued compute demand and total resident-cache pressure. The threshold of eight comes directly from P1. This is a deliberately simple admission baseline, not an oracle: it does not estimate reuse probability, prefix value, or optimal packing.

5 Results

5.1 Parked work is common and natively invisible

On a fixed eight-replica cluster, the simulated parked fraction is 60.2% at $(T, \tau) = (4, 4)$, 89.5% at $(8, 8)$, and 93.1% at $(16, 8)$. At those points, respectively 16.2%, 27.6%, and 24.3% of post-warm-up samples contain parked programs while running, GPU, and active-KV signals are all zero. Figure 3 distinguishes how much work is parked from how often its invisibility is complete.

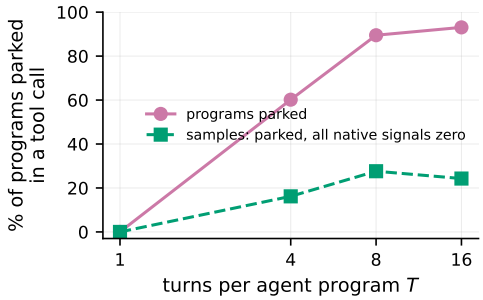

Figure 3: Simulated parked fraction and completely invisible parked intervals on a fixed cluster. Reclaimable cache is tracked internally but is not a native autoscaling signal.

5.2 Counting parked programs is not a robust policy

Table 1 reverses the uploaded paper's headline. ParkAware beats HPA on three of six agentic points and KV-util on two, but loses in the mean. It beats KEDA on five of six, yet usually spends more GPU time. Pressure-aware admission has the best mean SLO, but its 3.44× mean cost gives it the worst SLO/cost ratio. Predictive has the best mean pointwise SLO/cost ratio precisely because it tolerates substantial SLO loss. No dynamic policy simultaneously matches the cheapest static SLO configuration and costs less on average. Parked count alone and strict cache protection occupy opposite sides of an explicit reliability–cost tension.

The result varies by regime. At $T = 4, \tau = 8$, HPA attains 1.000 and ParkAware 0.693. At $T = 16, \tau = 30$,

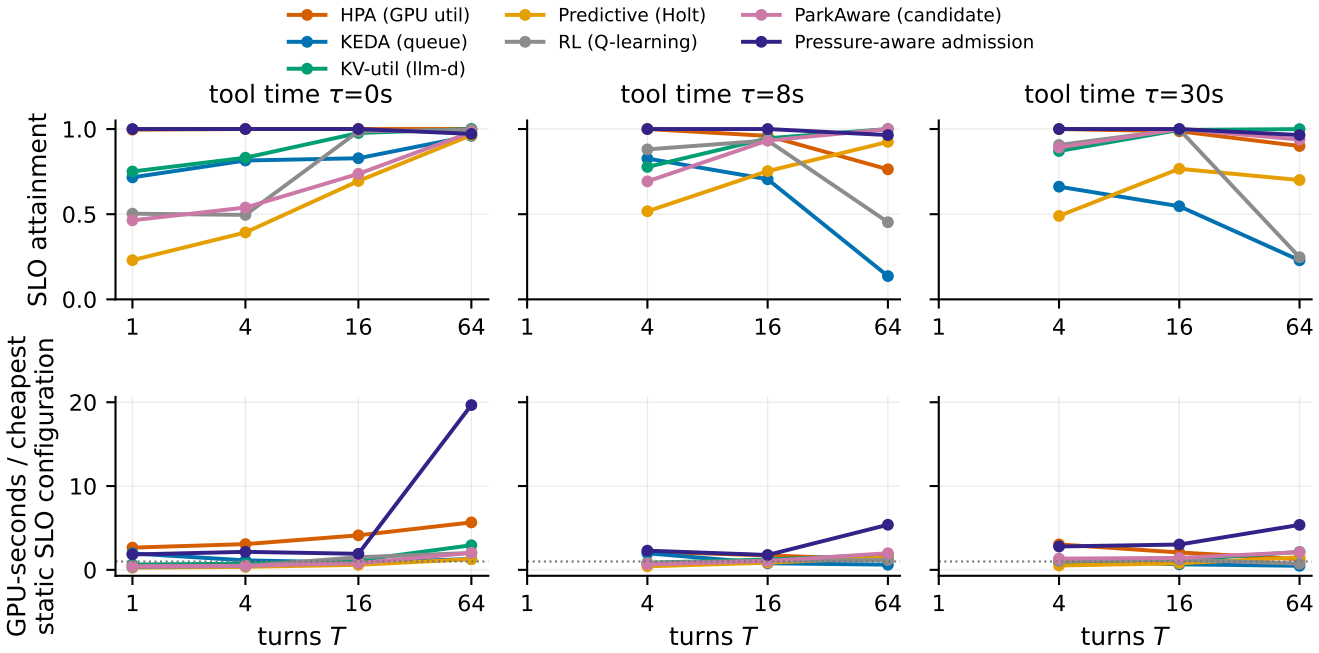

Figure 4: SLO attainment and GPU cost relative to the cheapest static SLO-meeting cluster. Pressure-aware admission protects SLO at most points but pays for that protection; no dynamic policy dominates on both axes.

Table 1: Mean over the six coding-trace agentic points ($T > 1, \tau > 0$).

Policy	SLO	GPU / static	SLO / cost
Pressure admission	**0.988**	3.438	0.341
HPA (GPU)	0.935	1.914	0.527
KEDA (queue)	0.518	0.956	0.565
KV-util	0.931	1.339	0.765
Predictive	0.692	0.925	**0.854**
RL (Q-learning)	0.736	1.017	0.720
RL + parked	0.653	0.922	0.706
ParkAware candidate	0.909	1.430	0.709

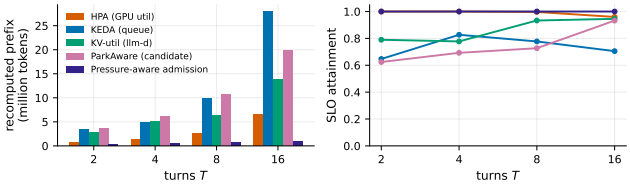

Figure 5: Recomputed prefix tokens and SLO on the coding trace. Recompute includes partial pressure eviction and cache loss on scale-in.

ParkAware attains 1.000 and HPA 0.987. At $T = 64, \tau = 8$, KV-util and ParkAware both attain 1.000 while HPA is 0.764 and pressure-aware admission is 0.965. Such crossovers are exactly what a single occupancy proxy should produce when prefix value depends on context size, return timing, cache pressure, and service batching.

5.3 Pressure dominates scale-in damage

At $T = 8, \tau = 8$, HPA recomputes 2.58M tokens, of which only 0.218M are attributed to scale-in. KEDA, KV-util, and ParkAware recompute 9.82M, 6.44M, and 10.76M tokens. Their scale-in components are only 0.532M, 0.163M, and 0.278M. Aggregated across these four baseline policies at $T = 2, 4, 8, 16$, pressure eviction accounts for 97.03% of destroyed prefix tokens and scale-in for 2.97%. The pressure share is 95.74%, 94.73%, 95.97%, and 98.28% at the four turn counts, so it is not a single-point artifact. This agrees with P1's independent hardware evidence: elapsed time barely moves survival while enough neighboring contexts erase it.

Pressure-aware admission eliminates pressure eviction at every E4 point. It does not eliminate scale-in loss, and strict protection costs replicas: at $T = 8$ it attains 1.000 SLO with 11.84k GPU-seconds versus HPA's 0.997 with 14.34k and KV-util's 0.933 with 5.74k. The old equation that charged every parked context only when a replica was removed therefore described the wrong dominant mechanism, while a controller that protects every surviving prefix can also overpay.

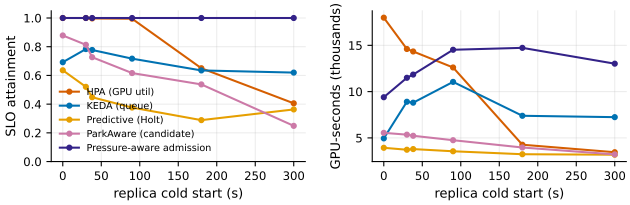

Figure 6: Cold-start sensitivity at $T = 8, \tau = 8$. The measured 14B point is 38.155 s; larger values are sensitivity cases, not measurements.

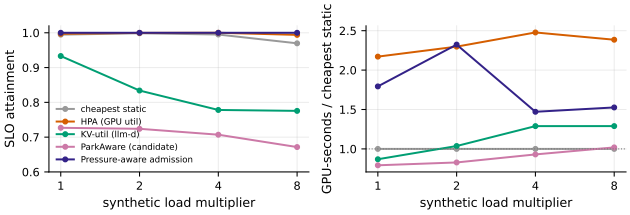

Figure 7: Synthetic load stress at $T = 8, \tau = 8$. Pressure-aware admission protects SLO with less GPU time than HPA at 4–8×, but remains above the cheapest static reference.

5.4 Cold start and workload change the ordering

At the measured 38.155 s process cold start, HPA, KEDA, Predictive, and ParkAware attain 0.997, 0.777, 0.448, and 0.727 at $(T, \tau) = (8, 8)$. At 180 s they attain 0.650, 0.635, 0.289, and 0.537. At 300 s KEDA attains 0.620, exceeding HPA's 0.406 and ParkAware's 0.249 because its conservative queue response avoids some extremely expensive reconstruction. The original monotone ParkAware advantage does not survive correction. Pressure admission holds 1.000 throughout the sweep, but at 300 s its p99 program latency reaches 1,005 s and its waste ratio reaches 0.758; SLO protection is not free.

On the conversation trace constructed as $T = 8, \tau = 8$, HPA attains 1.000, KV-util 0.910, and ParkAware 0.972; KEDA attains 0.503. On the coding trace at the same point, the values are 0.997, 0.933, 0.727, and 0.777, respectively. Pressure admission attains 1.000 on both. A program-level policy cannot be evaluated independently of the token and arrival distribution beneath the constructed program structure.

5.5 Sensitivity at realistic trajectory length

The original E7 used $T = 8$, where HPA leads the four original policies in all nine capacity and batch rows (with ties at batch 8). We rerun it at $T = 64$, closer to long agent trajectories. At measured capacity and batch 32, HPA attains 0.764, KV-util and ParkAware attain 1.000, and pressure admission attains 0.965. Across the $T = 64$ capacity sweep, the winner changes: KV-util and ParkAware lead at 31k, 62k, and measured capacity; pressure admission leads at 150k; and HPA, ParkAware, and pressure admission tie at 500k. Pressure admission ranges from 0.853 to 1.000. At $T = 64$, changing ParkAware's target from 0.5 to 0.9 no longer changes its 1.000 attainment, showing that a sensitivity conclusion at $T = 8$ does not generalize to longer trajectories.

5.6 High load exposes a useful pressure-control regime

The base trace is light: its cheapest static SLO configuration uses only 1–4 replicas over the main sweep. E8 deterministically superposes phase-shifted copies of the coding trace at 1, 2, 4, and 8× load. This is a synthetic stress test, not a measured production rate. At $(T, \tau) = (8, 8)$, the cheapest static reference grows from 3 to 5, 8, and 14 replicas. At 4× and 8×, pressure admission attains 1.000 SLO with 1.471 and 1.526× static GPU cost. HPA attains 1.000 and 0.994 with 2.477 and 2.385× cost, so pressure admission uses 40.6% and 36.0% fewer GPU-seconds than HPA. It still costs more than static, and its SLO/cost ratio does not dominate every lower-SLO policy. The positive result is therefore specific: pressure protection is a useful dynamic mechanism under load, not a universal cost optimum.

6 Implications

Expose cache state, not only allocation. Active-KV utilisation correctly reports allocated blocks, but an autoscaler also needs the amount, age, ownership, and eviction priority of reclaimable prefix content. Conflating the two would make admission control unsafe; exporting both would preserve the distinction.

Value is probabilistic. A parked prefix is worth approximately its miss penalty times the probability that the program returns before pressure evicts it. Tool type, transcript size, recent reuse, and cache competition may predict that value. A raw parked count assigns the same value to a 1k prefix and a 32k prefix despite a measured 56-fold difference in recompute penalty.

Admission is the first pressure-control lever. Scale-in is destructive when it targets a replica with valuable surviving prefixes, but it explains only 2.97% of baseline token loss here. Admission and placement act before the dominant event: a neighbor reclaiming those blocks. A practical controller should reject prefix-destructive placement, scale out when the rejected work is worth serving, and still allow low-value prefixes to be reclaimed. Our strict baseline implements the first two pieces and exposes the cost of omitting the third.

7 Threats to Validity

Single hardware configuration. Four modelling assumptions are measured on a single vLLM instance. One assumption was falsified by that measurement and the model corrected accordingly. Results may differ across engines, versions, models, cache managers, and hardware. V4 measures a process restart with warm host page cache, not a machine reboot or disk-cold start.

Constructed program structure. Azure supplies request records, not agent identities or tool intervals. Our T and τ are constructed. Sweeps and a $T = 1$ control expose, but do not eliminate, this limitation.

Simulator rather than cluster replay. Hardware measurements calibrate the mechanism and service curve; E2–E8 are still simulations. The model uses LRU pressure, one model per replica, and simplified placement. Production schedulers may migrate, preempt, offload, or explicitly pin cache state.

Synthetic high load. E8 superposes phase-shifted copies of one trace. It preserves within-copy request sizes and burst structure but is not an independent observation of a busier deployment. Its role is to test whether the low-load conclusion persists once static demand reaches 8–14 replicas.

Policy implementations. The policies follow their documented signal and standard control forms, but deployment-specific tuning can change results. The tabular RL baseline is deliberately conventional, not a claim about all learning-based controllers. We present ParkAware as a falsified candidate and pressure admission as a data-directed baseline, not production recommendations.

8 Related Work

PagedAttention and vLLM make KV management efficient within a serving instance [3]. Agent-serving systems increasingly schedule whole programs rather than isolated requests; Autellix, for example, uses cumulative program service time [4]. This work addresses the interface between that program state and cluster-level elasticity.

LLM autoscaling work optimizes token-aware or disaggregated serving. TokenScale scales on token velocity, while ServerlessLLM attacks model-loading cold start directly [5, 6]. DistServe and DynamoLLM optimize cluster organization and goodput [11, 12]. Our measurement shows that active request metrics omit a distinct object: reclaimable, application-owned prefix value between requests.

Classical elasticity separates reactive, proactive, and learning-based controllers [7, 9]. RL resource allocation has a long history [8]. Our corrected result is compatible with that literature's main lesson: policy quality depends on the state representation. Here, neither native active-resource signals nor a parked-program count represents prefix survival and value.

9 Conclusion

A parked agent does not hard-hold KV in native vLLM, but it may retain a cheap path back into execution. That path has measurable value, is invisible to native autoscaling metrics, and disappears under neighboring cache pressure. Correcting this distinction invalidates both our original mechanism and its claimed ParkAware advantage. Hardware and simulation then identify the same enemy: not elapsed parking time and rarely scale-in, but pressure from admitted work. A simple pressure-aware admission baseline eliminates simulated pressure eviction and becomes cheaper than HPA under high load, while its static and low-load costs expose the remaining value-estimation problem. Agentic autoscaling needs explicit reclaimable-prefix state, reuse value, and pressure-aware admission—not another idle-time scalar.

A Sensitivity Figure

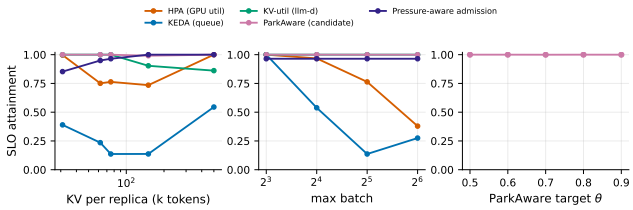

Figure 8: Sensitivity rerun at $T = 64, \tau = 8$: KV capacity, maximum batch, and the candidate ParkAware target.

References

[1] P. Patel et al. Splitwise: Efficient generative LLM inference using phase splitting. In *ISCA*, 2024. Traces: https://github.com/Azure/AzurePublicDataset.

[2] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[3] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

[4] M. Luo et al. Autellix: An efficient serving engine for LLM agents as general programs. arXiv:2502.13965, 2025.

[5] R. Lai et al. TokenScale: Timely and accurate autoscaling for disaggregated LLM serving with token velocity. arXiv:2512.03416, 2025.

[6] Y. Fu et al. ServerlessLLM: Low-latency serverless inference for large language models. In *OSDI*, 2024.

[7] N. Herbst, S. Kounev, R. Reussner. Elasticity in cloud computing: What it is, and what it is not. In *ICAC*, 2013.

[8] E. Barrett, E. Howley, J. Duggan. Applying reinforcement learning towards automating resource allocation and application scalability in the cloud. *Concurrency and Computation*, 25(12), 2013.

[9] Y. Garí et al. Reinforcement learning-based application autoscaling in the cloud: A survey. *Engineering Applications of Artificial Intelligence*, 2021.

[10] Polar: Agentic RL on any harness at scale. arXiv:2605.24220, 2026.

[11] J. Stojkovic et al. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *HPCA*, 2025.

[12] Y. Zhong et al. DistServe: Disaggregating prefill and decoding for goodput-optimized LLM serving. In *OSDI*, 2024.